

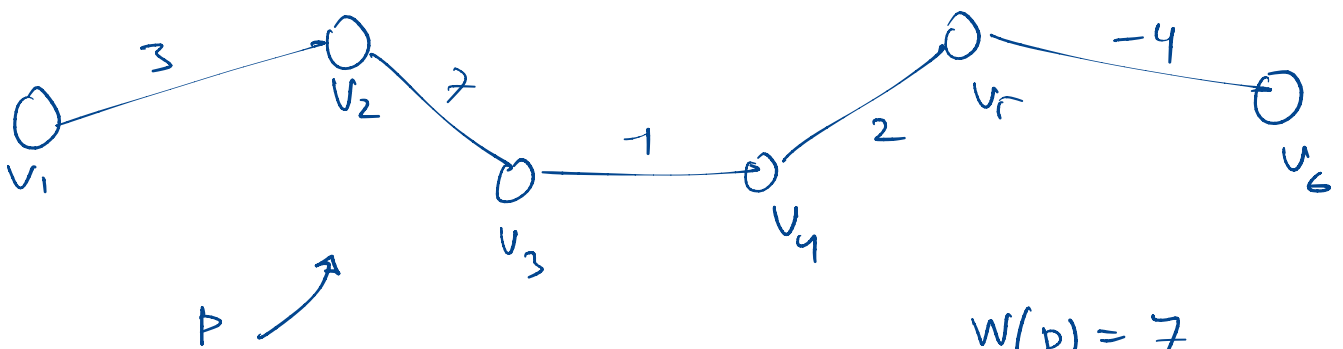
# Graph Algorithms - IV

## Shortest paths

Consider a weighted digraph  $G=(V,E)$  with edge weight function  $w(u,v):E \rightarrow \mathbb{R}$  for an edge  $(u,v) \in E$

The weight of a path  $P = \langle v_0, v_1, \dots, v_k \rangle$  is defined as

$$W(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$



$$\underline{W(P) = 7}$$

Shortest path: A shortest path from  $u$  to  $v$  is a path of minimum weight from  $u$  to  $v$ .

The shortest path weight is defined as:

$$\delta(u,v) = \begin{cases} \min \{ W(P) : u \xrightarrow{P} v \} & \text{if there is a path from } u \text{ to } v \\ \infty & \text{otherwise} \end{cases}$$

## Optimal Substructure

Theorem: A subpath of a shortest path is a shortest path

i.e., Given  $G = (V, E)$  a weighted digraph

Let  $P = \langle V_0, V_1, \dots, V_k \rangle$  be a shortest path

$V_0$  to  $V_k$ . Then for any  $i, j$  such that  $0 \leq i \leq j \leq k$  the subpath

$P_{ij} = \langle P_i, P_{i+1}, \dots, P_j \rangle$  is a shortest path between  $V_i$  &  $V_j$ .

Proof: Decompose path  $P$  into

$$V_0 \xrightarrow{P_{0i}} V_i \xrightarrow{P_{ij}} V_j \xrightarrow{P_{jk}} V_k$$

$$W(P) = W(P_{0i}) + W(P_{ij}) + W(P_{jk})$$

Let's assume that there is a shorter path  $P'_{ij}$  between  $P_i$  &  $P_j$  such that  $W(P'_{ij}) < W(P_{ij})$

In that case

$V_0 \xrightarrow{P_{0i}} V_i \xrightarrow{P'_{ij}} V_j \xrightarrow{P_{jk}} V_k$  is a  
Path between  $V_0$  &  $V_k$  with path length

$W(P_{0i}) + W(P'_{ij}) + W(P_{jk})$  smaller than

$W(P)$ . However we know that  $W(P)$   
is the minimal path length between  $V_0$  &  $V_k$ .

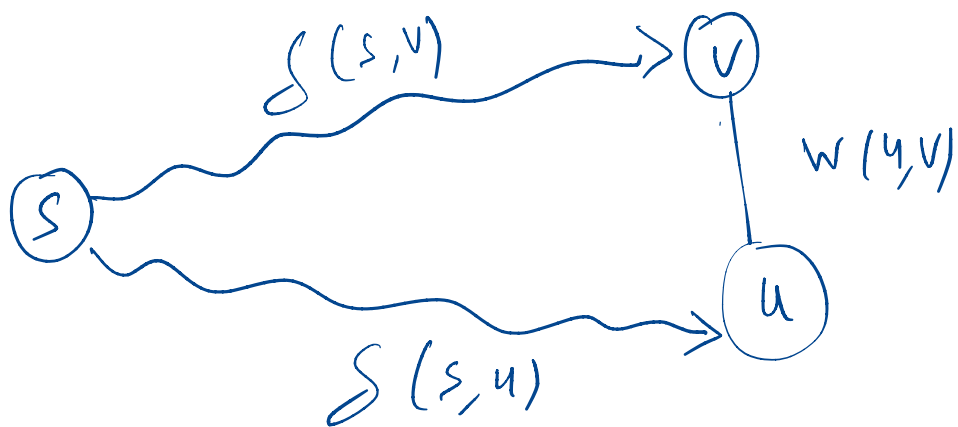
Therefore our assumption is incorrect  
and by contradiction  $P_{ij}$  is the shortest  
path between  $V_i$  &  $V_j$ .



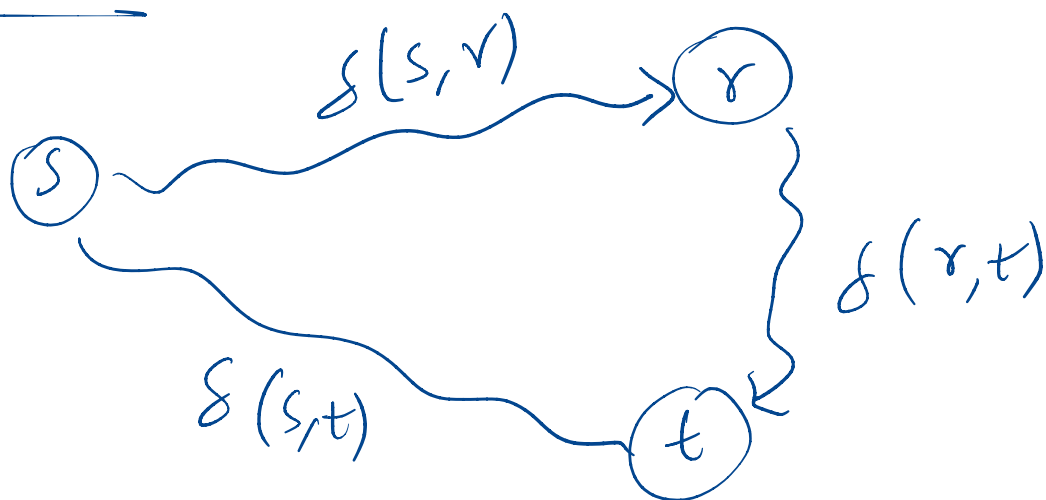
Themen: Triangle inequality for graphs

For a source vertex  $s \in V$  & for all edges  
 $(u, v) \in E$

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

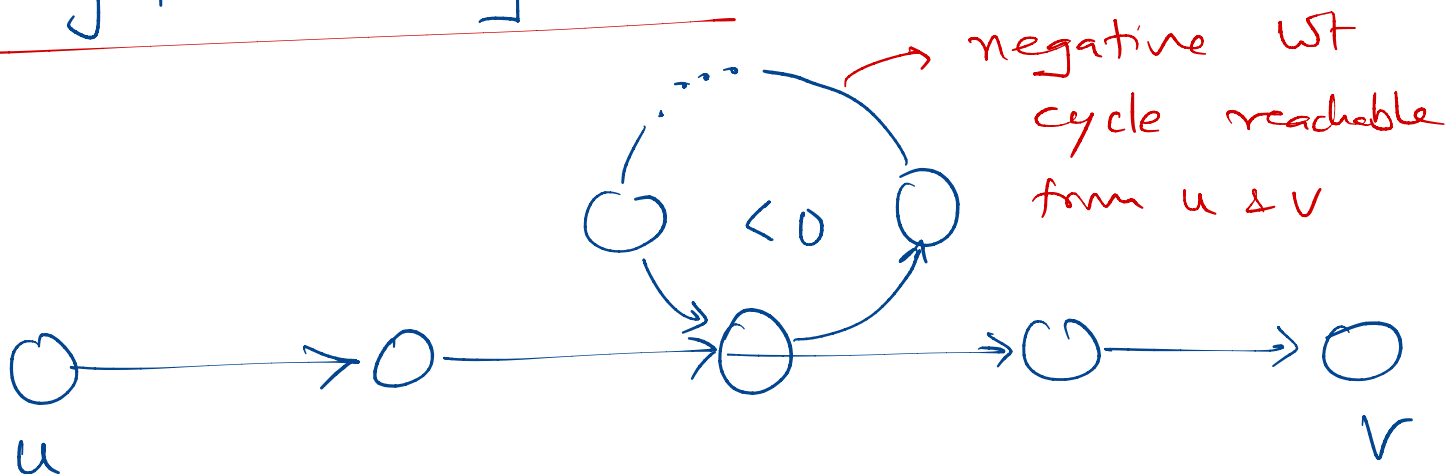


Extension:



$$g(s, r) \leq g(s, t) + g(r, t)$$

Negative weights



If a graph contains a negative wt. cycle then some shortest path may not exist.

→ Not a problem with DAG with negative weights.

## Single source Shortest Path

From a given source vertex  $s \in V$ , find shortest path weights  $\delta(s, v)$  to all  $v \in V$

↓  
if edge weights  $w(u, v)$  are non-negative then shortest paths must exist

### Greedy Approach:

- maintain a set  $S$  of vertices whose shortest path distances from  $s$  are known
- At each step add to  $S$  the vertex  $v \in V - S$  whose distance estimate from  $S$  is minimal

— Update distance estimates of vertices adjacent to  $v$ .

## Dijkstra's algorithm

Given digraph  $G = (V, E)$  with edge weight

$$w(u, v) : E \rightarrow \mathbb{R}$$

Maintain :  $S \rightarrow$  set of vertices in the shortest path tree

$Q \rightarrow$  Min priority queue containing vertices in the set  $V - S$ , keyed on path length estimates  $d$

# Dijkstra ( $G, w, s$ )

Shortest path  
distance estimate  
from  $s$

for each vertex  $v \in G.V$

$$\left| \begin{array}{l} v.d = \infty \\ v.\pi = NIL \end{array} \right.$$

Initialization

$$s.d = 0$$

$$S = \emptyset$$

$$Q = \emptyset$$

for each vertex  $u \in G.V$

    Insert ( $Q, u$ )

$O(E \lg V)$

while  $Q \neq \emptyset$

$u = \text{Extract-min}(Q)$

$$S = S \cup \{u\}$$

for each vertex  $v$  in  $G.\text{Adj}[u]$

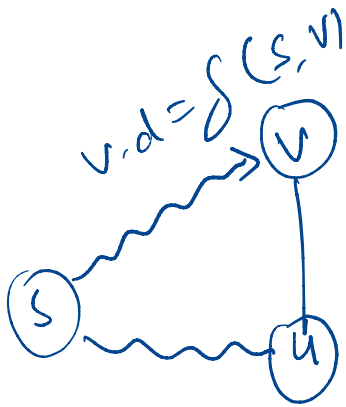
    if  $v.d > u.d + w(u, v)$

$$v.d = u.d + w(u, v)$$

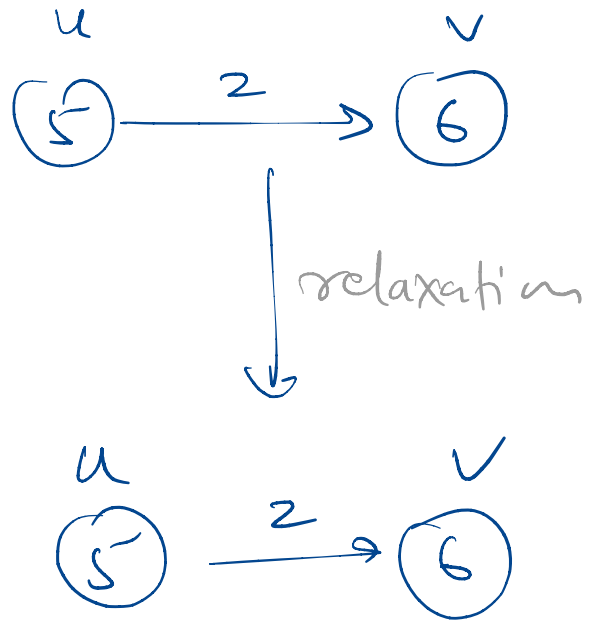
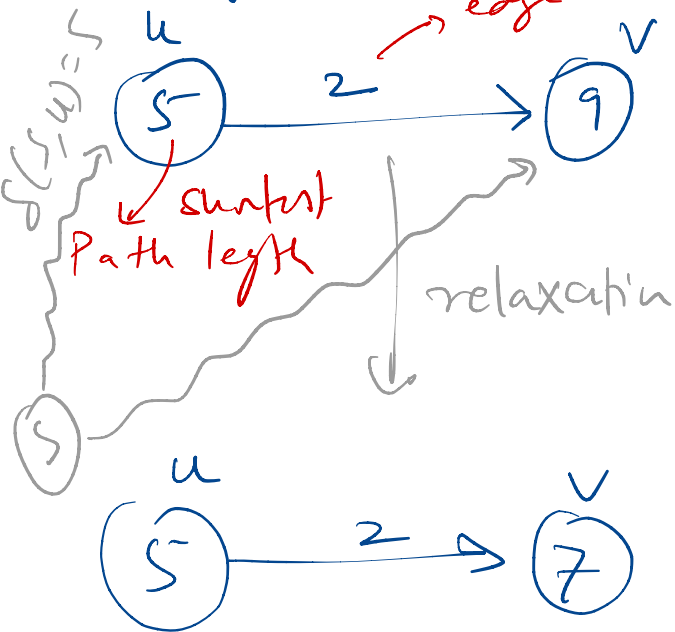
$$v.\pi = u$$

    Decrease-key ( $Q, v, v.d$ )

Edge relaxation



## Edge relaxation



## Example:

